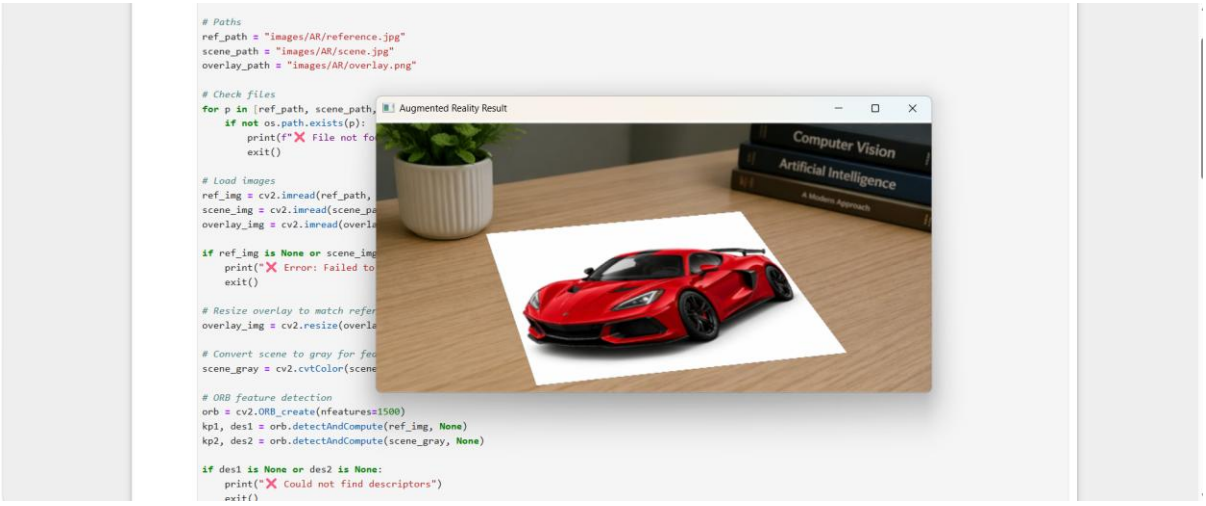
exp 10

```
import cv2
import numpy as np
ref_img = cv2.imread("images/AR/reference.jpg", 0) #
query image (grayscale)
scene_img = cv2.imread("images/AR/scene.jpg")
overlay_img = cv2.imread("images/AR/overlay.png")
if ref_img is None or scene_img is None or
overlay_img is None:
print("Error: Check image paths")
exit()
overlay_img = cv2.resize(overlay_img,
(ref_img.shape[1], ref_img.shape[0]))

orb = cv2.ORB_create(nfeatures=1000)
kp1, des1 = orb.detectAndCompute(ref_img, None)
kp2, des2 =
orb.detectAndCompute(cv2.cvtColor(scene_img,
cv2.COLOR_BGR2GRAY),
```

```
None)
bf = cv2.BFMatcher(cv2.NORM_HAMMING,
crossCheck=True)
matches = bf.match(des1, des2)
matches = sorted(matches, key=lambda x: x.distance)
good_matches = matches[:50]
if len(good_matches) < 10:
print("Not enough matches found")
exit()
src_pts = np.float32([kp1[m.queryIdx].pt for m in
good_matches]).reshape(-1,1,2)
dst_pts = np.float32([kp2[m.trainIdx].pt for m in
good_matches]).reshape(-1,1,2)
H, mask = cv2.findHomography(src_pts, dst_pts,
cv2.RANSAC, 5.0)
h, w = ref_img.shape
overlay_warp = cv2.warpPerspective(overlay_img, H,
(scene_img.shape[1],
scene_img.shape[0]))
```

```
mask_overlay = np.zeros(scene_img.shape, dtype=np.uint8)
mask_overlay[overlay_warp > 0] = 255
result = cv2.addWeighted(scene_img, 1, overlay_warp, 1, 0)
cv2.imshow("Augmented Reality Result", result)
cv2.waitKey(0)
cv2.destroyAllWindows()
```